

Phase 8 Adaptive Heuristic Portfolio Protocol

This document defines the next thesis-level phase after the phase-6 replay-mechanism study and the phase-7 modular-operator study.

The framing question is:

Can LLM-guided evolution discover instance-adaptive hyper-heuristics that select, schedule, and tune known routing heuristics better than a fixed baseline, random portfolio, or conventional learned selector?

The target is an interpretable algorithm-selection or hyper-heuristic controller over a frozen library of known components, rather than open-ended invention of a new TSP algorithm.

Scientific Motivation

Phase 7 showed that recurring operator families can reappear under search, but those operators did not survive the full transplant, ablation, and Pareto validation pipeline. That does not make the search useless. It suggests the stronger signal may be:

- restart control
- perturbation scheduling
- scaffold or heuristic selection
- candidate pruning

That is a classic hyper-heuristic or algorithm-selection problem rather than an open-ended algorithm-invention problem.

Research basis:

- Rice, The Algorithm Selection Problem ([https://doi.org/10.1016/S0004-3702\(76\)80025-4](https://doi.org/10.1016/S0004-3702(76)80025-4)), 1976
- Kerschke et al., Algorithm selection on a meta level (<https://link.springer.com/article/10.1007/s10994-022-06161-4>), 2023
- Drake et al., Hyper-heuristics: A survey and taxonomy (<https://www.sciencedirect.com/science/article/pii/S0360835223008392>), 2024
- Goutam et al., On automatic algorithm configuration of vehicle routing problem solvers (<https://link.springer.com/article/10.1007/s41604-019-00010-9>), 2019
- Lin et al., Landscape-Aware Bandit Hyper-Heuristics for Online Operator Selection in UAV Inspection Routing (<https://arxiv.org/abs/2605.14620>), 2026

Frozen Portfolio

Phase 8 freezes a library of known components and asks the LLM to evolve a controller, not a full solver.

Frozen component library:

- nearest neighbor
- cheapest insertion
- farthest insertion
- 2-opt
- limited 3-opt

- double-bridge perturbation
- random restart
- candidate-list pruning
- cluster-first route construction
- edge-preserving perturbation
- simple simulated-annealing acceptance
- multi-start local search

Concrete frozen heuristics are built as named recipes over those components:

- nearest_neighbor_multistart
- cheapest_insertion_two_opt
- farthest_insertion_two_opt
- candidate_pruned_two_opt
- limited_three_opt
- cluster_first_local_search
- edge_preserving_restart
- annealed_multi_start

The controller may select among these frozen heuristics and apply bounded schedule adjustments. It may not invent new low-level operators.

Controller Inputs

The controller sees instance descriptors:

- number of cities
- coordinate spread
- distance distribution statistics
- nearest-neighbor distance mean, variance, and CV
- MST length
- convex-hull ratio
- clustering score and cluster separation
- grid-likeness
- bottleneck or two-cluster score
- corridor or elongated-structure score
- nearest-neighbor trap score

It may also condition on lightweight online search-state features:

- stagnation length
- recent improvement rate
- current tour length

- number of accepted swaps
- time budget used
- edge-length skew in the current tour
- failed perturbation count

These features are low-dimensional and interpretable so the final controller can be inspected directly.

Controller Output

The output is one deterministic Python function:

```
def build_controller():
    return {
        "name": "...",
        "description": "...",
        "default_heuristic": "...",
        "random_like_heuristic": "...",
        "clustered_heuristic": "...",
        "grid_like_heuristic": "...",
        "two_cluster_bottleneck_heuristic": "...",
        "corridor_heuristic": "...",
        "nearest_neighbor_trap_heuristic": "...",
        "stagnation_switch_heuristic": "...",
        "stagnation_threshold": 3,
        "low_improvement_threshold": 0.08,
        "failed_perturbation_threshold": 2,
        "time_budget_trigger": 0.75,
        "candidate_limit_offset": 0,
        "restart_offset": 1,
        "temperature_scale": 1.0,
        "acceptance_bias": 0.0,
    }
```

This is a bounded hyper-heuristic interface. It is expressive enough to model selection, scheduling, and mild tuning, but not to hide a full solver rewrite.

Condition-specific execution semantics:

- `llm_static_selector` is instance-conditioned only. It selects and tunes one frozen heuristic from instance descriptors and does not use online search-state switching.
- `llm_evolved_adaptive_controller` and `llm_evolved_controller_diversity_failure_replay` may use both instance descriptors and the lightweight online search-state features listed below.

Comparison Arms

The official condition set is:

1. `best_single_fixed_heuristic`
2. `random_portfolio`
3. `oracle_selector`
4. `supervised_ml_selector`
5. `llm_static_selector`
6. `llm_evolved_adaptive_controller`
7. `llm_evolved_controller_diversity_failure_replay`
8. `full_solver_evolution`

Interpretation:

- best_single_fixed_heuristic: main fixed baseline
- random_portfolio: dumb portfolio control baseline
- oracle_selector: per-instance upper bound inside the frozen portfolio
- supervised_ml_selector: non-LLM learned baseline using the same descriptors
- llm_static_selector: one-shot interpretable rule baseline
- llm_evolved_adaptive_controller: main phase-8 condition
- llm_evolved_controller_diversity_failure_replay: replay-aware controller condition
- full_solver_evolution: strong opaque baseline carried forward from earlier routing phases

The oracle is not a deployable method. It defines the attainable ceiling of the frozen portfolio on the evaluation set.

Oracle Regret

The defining phase-8 secondary metric is:

$\text{selector_regret} = \text{achieved_gap} - \text{oracle_portfolio_gap}$

This makes the question clean:

Can the controller approach the oracle portfolio on held-out instances without seeing those instances during training?

Endpoints

Primary endpoint:

- held-out TSPLIB mean optimality gap

Secondary endpoints:

- selector regret versus the oracle portfolio
- runtime-adjusted gap
- Pareto efficiency over gap, runtime, and complexity
- cross-family transfer on synthetic validation families
- accepted code novelty
- controller complexity
- adaptation efficiency

The strongest positive phase-8 result shape is not necessarily the absolute lowest gap. A valid success pattern is:

- held-out gap comparable to full-solver evolution
- lower runtime inflation than full-solver evolution
- lower complexity than full-solver evolution
- interpretable instance-conditioned rules that are stable across replicated runs

Interpretation Rules

- Do not claim algorithm invention from phase 8. The controller is selecting and scheduling known heuristics.
- Do not treat oracle performance as directly deployable. It is an upper bound.
- Prefer claims about regret reduction, runtime-adjusted performance, and interpretability over raw held-out gap alone.
- If the LLM controller beats the fixed baseline but not the supervised selector, report that honestly as partial evidence for interpretability rather than as a full LLM-specific win.
- If replay helps the controller, interpret that as evidence about controller training stability or coverage, not about invention of new routing primitives.

Replication

Official main-study target:

- 20 paired seed offsets with the fixed list 0, 1000, 2000, ..., 19000

Use paired comparisons across the same offsets and report uncertainty on paired deltas, not only per-condition means.

Deliverable

The required deliverable is a short adaptive-portfolio report that answers:

1. Can the LLM-derived controller beat the best fixed heuristic on held-out TSPLIB?
2. How close does it get to the oracle portfolio?
3. Does it beat the supervised non-LLM selector?
4. Does replay improve the adaptive controller?
5. Does the controller deliver a useful gap/runtime/complexity tradeoff relative to full-solver evolution?
6. Are the learned rules interpretable and instance-adaptive rather than just opaque tuning?

Operational Entry Points

- Prepare the benchmark manifest with `prepare_tsp_benchmarks.py`
- Run the suite with `run_tsp_phase8_suite.py` and `configs/tsp_phase8_suite/01_adaptive_portfolio.json`
- Use `configs/tsp_phase8_suite/RUNBOOK.md` for concrete commands
- Aggregate repeated runs with `aggregate_tsp_phase8_runs.py`
- The official phase-8 evidence readout is tracked in `docs/PHASE_8_ADAPTIVE_HEURISTIC_PORTFOLIO_RESULTS_2026-05-21.md`

Relationship To Phase 7

Phase 7 asked whether the search could discover a reusable low-level TSP operator. Phase 8 shifts the goal:

- from operator invention
- to hyper-heuristic control over a fixed, known portfolio

That is the conservative, higher-probability closeout path because it matches both the observed rediscovery patterns from phase 7 and the algorithm-selection literature.